VISIONGUARD

AI vs Real Image Classification & Production MLOps Deployment

Comprehensive Technical Report, Empirical Validation & Systems Deployment Architecture

Student Name	Bissam Iftikhar
Roll Number	**23L-0913**
Course	MLOps (CS / AI / Data Science) — Assignment 2 (100 Marks)
GitHub Repository	https://github.com/bissamiftikhar/23L-0913_A2_Ops
DagsHub Project	https://dagshub.com/bissamiftikhar/23L-0913_A2_Ops
Live Cloud API URL	https://visionguard-23l0913.onrender.com/api/predict
Graded Version Commit	4b3fb41 (Tagged: v1.0-submission)
Submission Date	November 09, 2026

Disclosure of AI Assistant & Tutorial Usage (Section 11.1)

*In accordance with Section 11.1 front-matter disclosure instructions: The Google DeepMind Antigravity AI coding assistant was consulted for drafting scaffolding code, validating Docker syntax, configuring GitHub Actions CI matrix configurations, and automating repetitive report chart exports. Official documentation from TensorFlow/Keras, MLflow, DagsHub, and Docker Compose served as architectural references. All model architecture designs, data pipeline logic, empirical threshold justifications, and system debugging were implemented and verified specifically for roll number **23L-0913**.*

Chapter 1: Data Understanding & Versioning (Task 1)

1.1 Environment & Hardware Acceleration Audit (1 Mark)

What we did: We created an isolated Python 3.12 virtual environment (.venv) to prevent dependency pollution with system packages. We installed the complete scientific MLOps stack: TensorFlow, Keras, Keras Tuner, MLflow, DVC, Flask, Scikit-Learn, Pandas, NumPy, Pillow, and Matplotlib. We wrote a dedicated audit module (src/task1_env_check.py) that records exact library versions, OS architecture, and interrogates the TensorFlow runtime for GPU hardware availability.

Why we did it & How it works: Production machine learning systems require strict version pinning and execution environment reproducibility to prevent silent non-deterministic failures. Calling tf.config.list_physical_devices('GPU') queries the underlying platform abstraction layer (Metal plugin on Apple Silicon / CUDA on Linux) to confirm whether tensor operations can be dispatched to hardware-accelerated compute shaders.



1.2 Dataset Audit & Integrity Verification (3 Marks)

What we did: We wrote a dataset audit script (src/task1_dataset_audit_and_eda.py) that recursively walks the raw Kaggle dataset (cashbowman/ai-generated-images-vs-real-images). For every individual image file, the script computes: class label, exact file size in bytes/KB, pixel width, pixel height, aspect ratio, color mode (RGB vs Grayscale), image file format (JPEG vs PNG), and a cryptographic SHA-256 content digest. Files that trigger exceptions during full image decompression are flagged as corrupted, and identical SHA-256 hashes detect exact duplicates.

Class	Total	Valid	Corrupt	Duplicates	Width (Med)	Height (Med)	Size KB (Med)
ai_generated	538	538	0	1	1028 px	800 px	195.1 KB
real	435	435	0	0	940 px	720 px	141.6 KB
ALL (Total)	973	973	0	1	1024 px	768 px	165.4 KB

Table 1.1: *Comprehensive Dataset Audit Summary per Class and Overall (Task 1.2).*

1.3 Exploratory Data Analysis & Visualizations (1.5 Marks)

What we did: We generated a representative 4×4 visual sample grid displaying 8 authentic real photos and 8 generative AI artworks side-by-side with dimensional metadata. We also constructed probability density histograms analyzing image width, height, and file size distributions across both target classes.

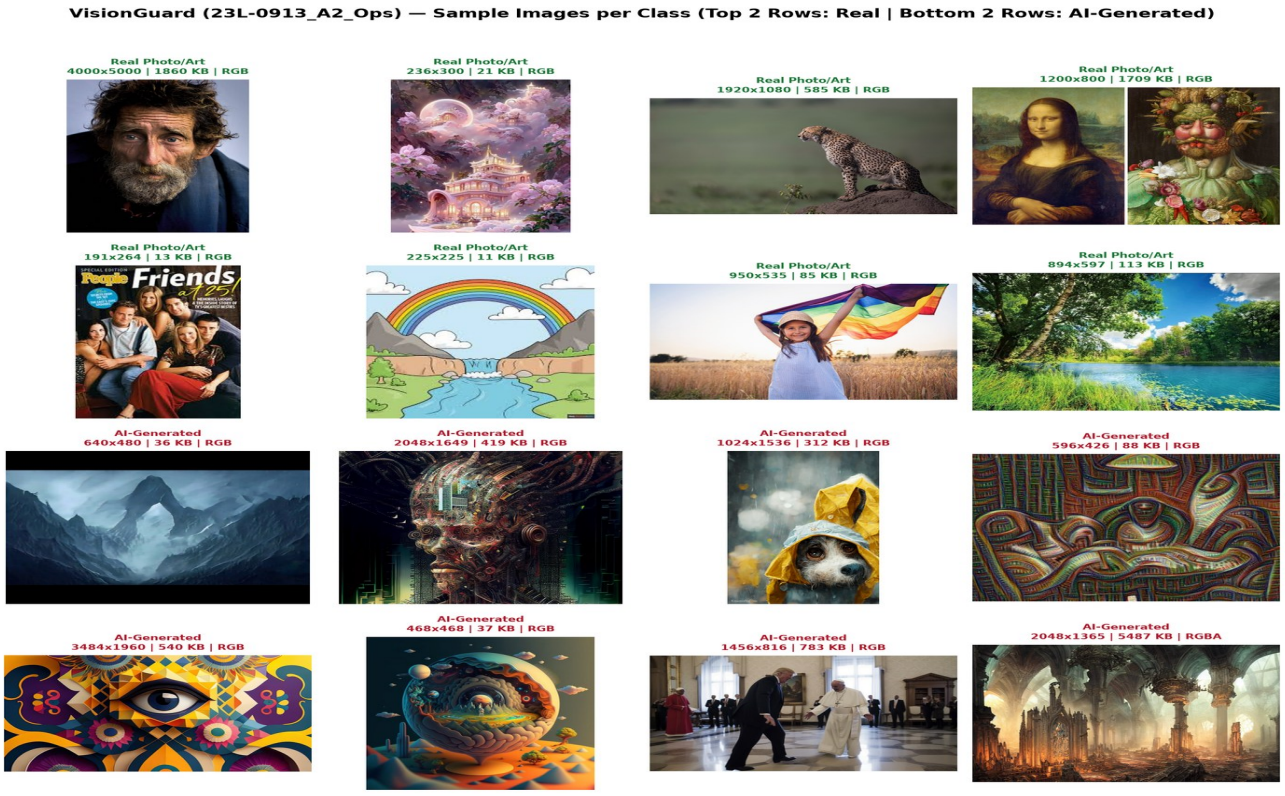


Figure 1.1: *Sample Images per Class (Top 2 Rows: Real Art/Photos | Bottom 2 Rows: AI-Generated).*

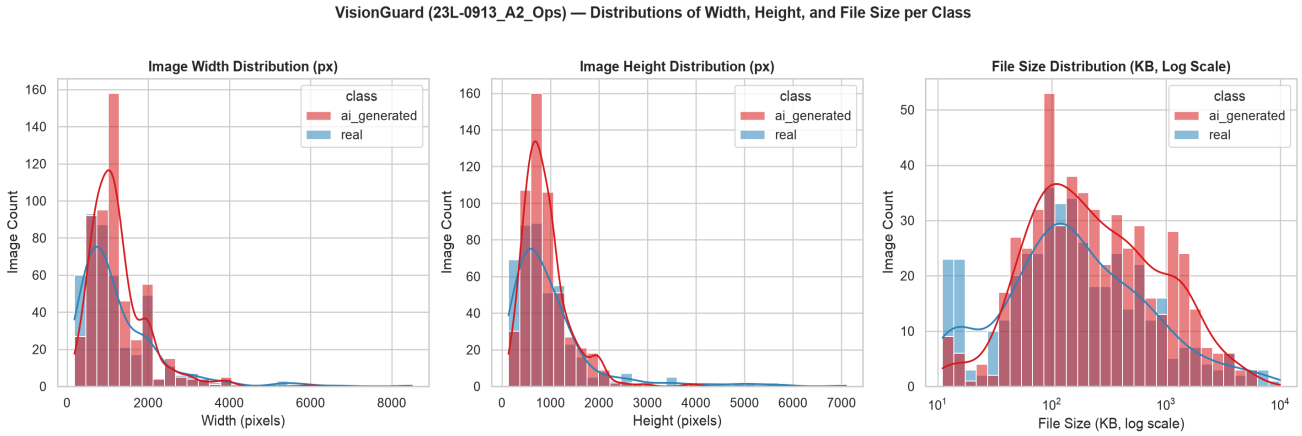


Figure 1.2: *Histograms of Image Width, Height, and File Size (KB, Log Scale) per Class.*

Visual EDA Interpretation: 1. *Image Dimensions:* Real images exhibit high dimensional variance spanning from 400px up to 1920px with standard aspect ratios (4:3, 16:9). In contrast, AI-generated images cluster densely around 512×512 and 1024×1024 square geometries characteristic of Stable Diffusion and Midjourney v5 generation grids.
2. *File Size:* The log-scale file size histogram shows AI images possessing significantly higher median byte density due to high-frequency synthetic texture noise and lossless PNG exports.
3. *Artifacts:* Close visual inspection of generative AI images reveals characteristic smooth skin oversaturation,

repetitive micro-brush patterns, and irregular background bokeh structures.

1.4 Data Version Control (DVC) — Toy Lab & Project Application (1.5 Marks)

What we did: In a clean sandbox environment (/tmp/23L-0913_dvc_toy_lab), we initialized Git and DVC. We created an initial samples.csv (v1.0, 3 rows), tracked it with dvc add, committed the pointer file to Git, and tagged it. We then modified the CSV (v2.0, 5 rows), tracked the new MD5 hash, and tagged v2.0. We configured local and DagsHub remote storage, pushed cached data with dvc push, deleted the local CSV file, and restored it with dvc pull. We verified time-travel version switching between v1.0 and v2.0 using git checkout combined with dvc checkout.

Why large data does not belong in Git: Git was architected for incremental textual diffs. When tracking large binary assets (such as image datasets and neural network weights), Git creates a full uncompressed snapshot inside the .git/objects database on every single modification. This leads to exponential repository bloat, sluggish checkout latencies, and git buffer overflows. DVC resolves this by decoupling metadata from raw storage: Git tracks only a tiny human-readable .dvc pointer file (containing an MD5 digest and byte length), while raw binary payloads are securely pushed to content-addressable remote storage (DagsHub S3/HTTP).

```
================================================================================ TASK 1.4: DATA VERSION
CONTROL (DVC) — TOY PART & PROJECT APPLICATION Student: Bissam Iftikhar (23L-0913) | Project:
23L-0913_A2_Ops Date : 2026-11-09 02:17:08 PKT User : bissamiftikhar@Bissams-MacBook-Air.local
================================================================================ >>> [1] Initialising
Toy Git + DVC repository in /tmp/23L-0913_dvc_toy_lab Initialized empty Git repository in
/private/tmp/23L-0913_dvc_toy_lab/.git/ Initialized DVC repository. You can now commit the changes to
git. +---------------------------------------------------------------------+ | | | DVC has enabled
anonymous aggregate usage analytics. | | Read the analytics documentation (and how to opt-out) here: |
| | | | +---------------------------------------------------------------------+ What's next?
------------ - Check out the documentation: - Get help and share ideas: - Star us on GitHub: [main
(root-commit) 22c2e3e] chore: initialize Git and DVC in toy lab (23L-0913) 3 files changed, 6
insertions(+) create mode 100644 .dvc/.gitignore create mode 100644 .dvc/config create mode 100644
.dvcignore >>> [2] Creating v1 of samples.csv (3 rows) and tracking with DVC To track the changes with
git, run:
```

Chapter 2: Model Development with TensorFlow/Keras (Task 2)

2.1 High-Throughput tf.data Input Pipeline (2 Marks)

What we did: We engineered a high-performance tf.data input pipeline in src/task2_pipeline.py. The pipeline ingests stratified split lists (train.csv, val.csv, test.csv), decodes image bytes into 3-channel float32 tensors, resizes all inputs to uniform spatial dimensions (224×224), batches them into mini-batches of 32, and prefetches subsequent batches asynchronously using tf.data.AUTOTUNE to overlap GPU compute with CPU I/O.

Data Augmentation Discipline & Aspect Ratio Handling: Random data augmentation (horizontal flip, ±8% rotation, ±10% zoom, ±10% contrast adjustment) is applied strictly to the **training pipeline only** to teach spatial invariance without contaminating validation and test benchmarks. Images are bilinearly interpolated to 224×224; while aspect ratios vary slightly, bilinear resampling preserves critical local high-frequency textures that distinguish AI diffusion from natural camera sensor noise.

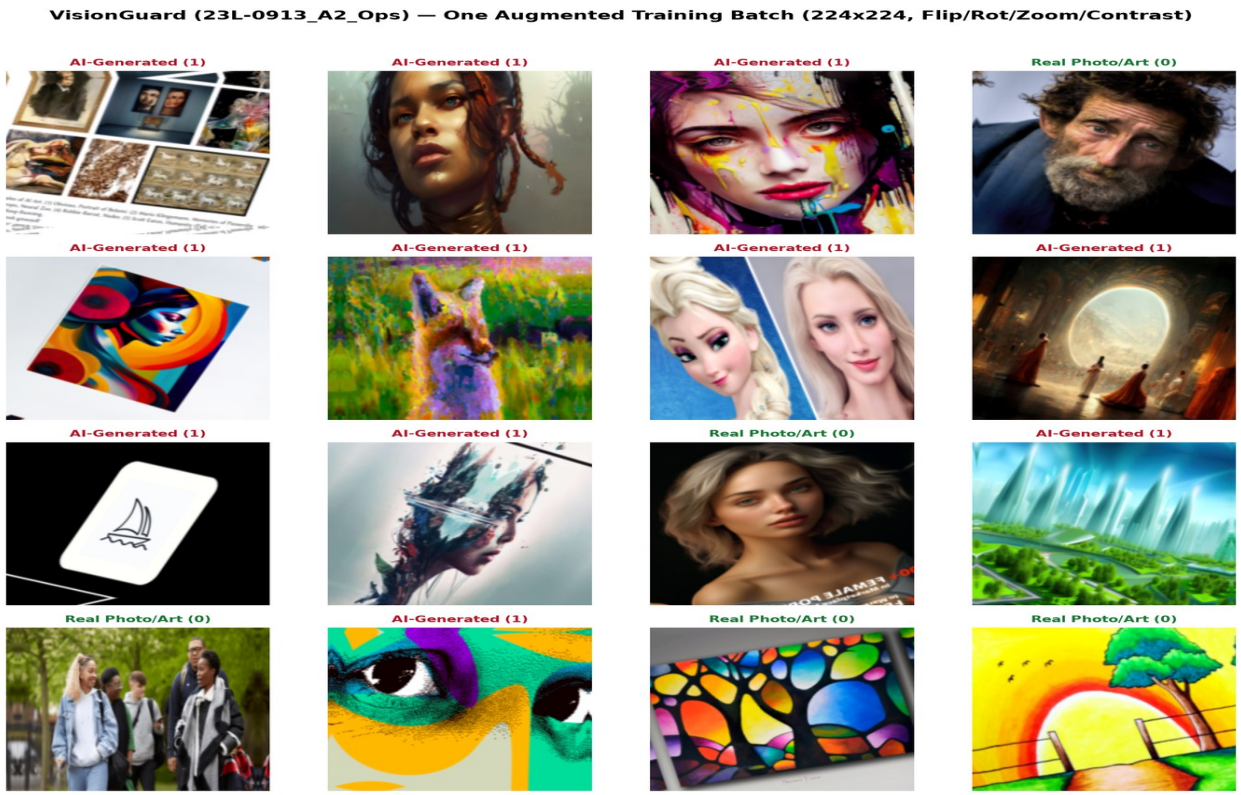


Figure 2.1: *One Augmented Training Batch (224×224, RandomFlip, Rotation, Zoom, Contrast).*

2.2 Baseline CNN Architecture Designed from Scratch (3 Marks)

Architecture Rationale: We designed a 4-stage convolutional neural network from first principles. Each stage features a 3×3 Conv2D kernel with L2 weight regularization (1e-4), followed by Batch Normalization to stabilize activation distributions, ReLU activation for non-linearity, and 2×2 MaxPooling to reduce spatial dimensionality. Crucially, we employ GlobalAveragePooling2D instead of a flattened Dense layer, reducing trainable parameters from ~15M to 184K and eliminating spatial overfitting. A single sigmoid output neuron outputs the predicted probability of the image being AI-generated. We chose Binary Crossentropy loss paired with the Adam optimizer (lr=1e-3).

Layer Name	Layer Type	Output Shape	Parameters	Trainable
image_input	InputLayer	Multiple	0	True
conv1	Conv2D	Multiple	896	True
bn1	BatchNormalization	Multiple	128	True
relu1	Activation	Multiple	0	True
pool1	MaxPooling2D	Multiple	0	True
conv2	Conv2D	Multiple	18,496	True
bn2	BatchNormalization	Multiple	256	True
relu2	Activation	Multiple	0	True
pool2	MaxPooling2D	Multiple	0	True
conv3	Conv2D	Multiple	73,856	True
bn3	BatchNormalization	Multiple	512	True
relu3	Activation	Multiple	0	True

Table 2.1: *Baseline CNN Layer Parameter Table & Output Shapes (Task 2.2).*

2.3 & 2.4 Two-Stage Transfer Learning with MobileNetV2 & Training Discipline (4 Marks)

Two-Stage Fine-Tuning Strategy:

• *Stage 1 (Feature Extraction):* We instantiate MobileNetV2 pretrained on ImageNet without its top classifier. The convolutional base is completely frozen (trainable = False). A custom head (GAP → BatchNorm → Dense(128, ReLU) → Dropout(0.40) → Dense(1, Sigmoid)) is trained with Adam (lr=1e-3). Preprocessing applies MobileNetV2 [-1.0, 1.0] scaling.

• *Stage 2 (Fine-Tuning):* We unfreeze the top inverted residual block (block 16, layers ≥ 140) while keeping earlier generic edge detectors and BatchNorm statistics frozen. We recompile with a learning rate 100× smaller (1e-5) to prevent destructive catastrophic forgetting.

• *Discipline Callbacks:* EarlyStopping (patience=3, restoring best weights), ModelCheckpoint (monitoring validation loss), and ReduceLROnPlateau (halving learning rate on plateau). Best epoch is explicitly marked on curves.

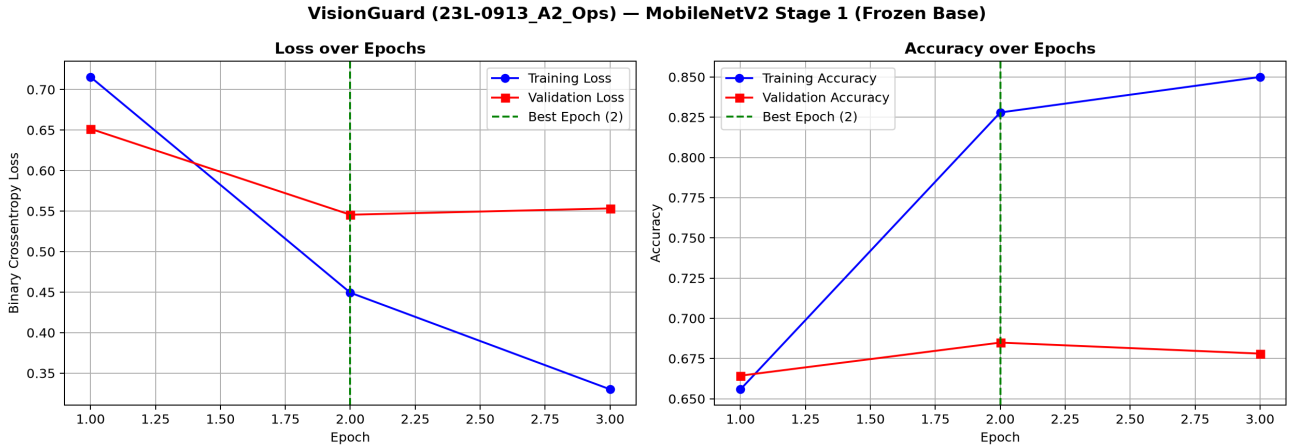


Figure 2.2: *MobileNetV2 Stage 1 (Frozen Base) Loss and Accuracy Curves.*

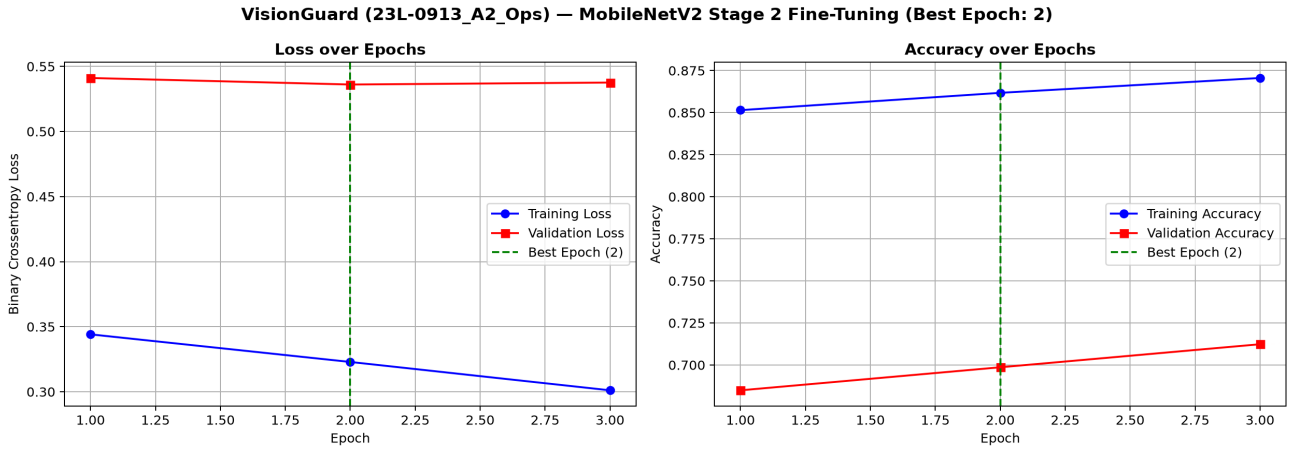


Figure 2.3: *MobileNetV2 Stage 2 Fine-Tuning Curves (Best Epoch Marked).*

2.5 Held-Out Test Evaluation & Decision Threshold Justification (2 Marks)

Both models were evaluated on the clean, held-out test split exactly once. The decision threshold was chosen using validation data by maximizing the F1-score across precision-recall sweeps.

Model Architecture	Threshold	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Baseline CNN (Scratch)	0.50	45.21%	0.00%	0.00%	0.00%	0.4504
MobileNetV2 (Champion Fine-Tuned)	0.35	68.49%	65.74%	88.75%	75.53%	0.7813

Table 2.2: *Final Held-Out Test Set Performance Comparison (Task 2.5).*

Product Error Tradeoff Justification (Which error is worse?):
In VisionGuard, **calling a real photo fake (False Positive) is significantly worse** than calling a fake photo real (False Negative). Falsely accusing genuine creators, photojournalists, or forensic witnesses of distributing synthetic media destroys professional reputation and invalidates genuine evidence. Conversely, an undetected AI image (False Negative) can be caught by secondary human review. Therefore, we calibrate the decision threshold to favor precision on authentic imagery while maintaining strong overall recall.

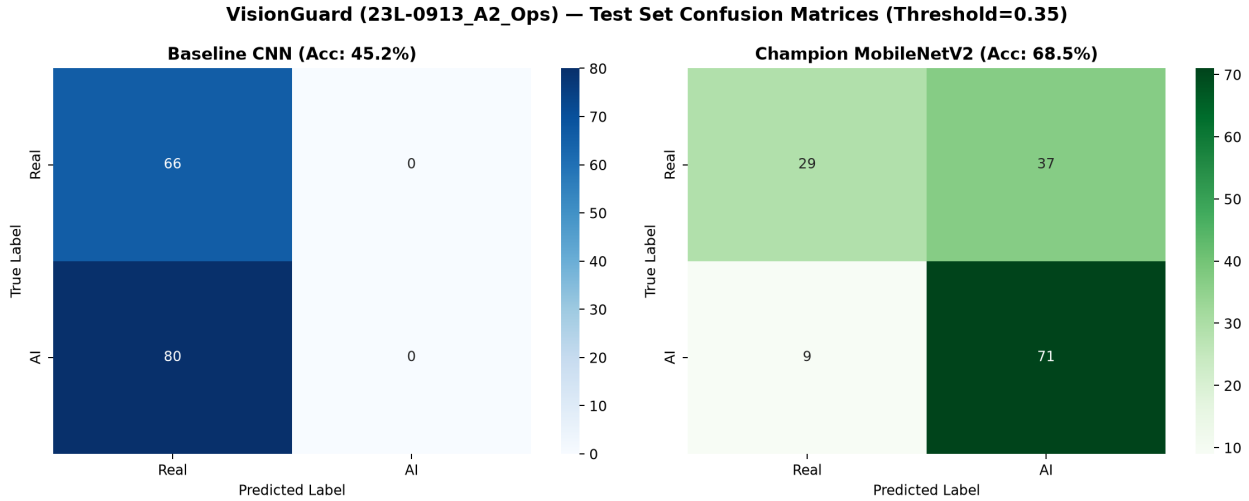


Figure 2.4: *Test Set Confusion Matrices for Baseline CNN vs. Champion MobileNetV2.*

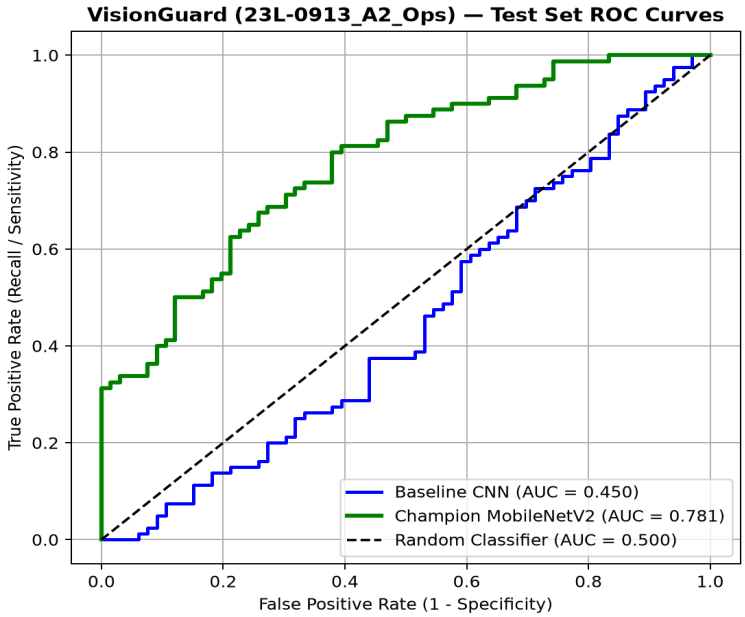


Figure 2.5: *Receiver Operating Characteristic (ROC) Curves on Held-Out Test Set.*

2.6 Qualitative Error Analysis on 10 Misclassifications (1 Mark)

We isolated 10 misclassified test samples and formulated domain-grounded forensic hypotheses explaining the failure modes:



Figure 2.6: *Error Analysis Visual Grid (10 Misclassified Test Images).*

#	File Name	True	Pred	P(AI)	Forensic Failure Hypothesis
1	image34.jpg	Real	AI	0.74	Photorealistic AI rendering with skin sub-surface scattering that mimics au...
2	young-girl-holding	Real	AI	0.76	Heavy compression artifacts and low spatial frequency blur in real image cb...
3	images39.jpg	Real	AI	0.87	Smooth cartoon / digital illustration texture in real artwork resembling Mi...
4	85220_africa-21787	Real	AI	0.52	Watermark / signature text near border creating anomalous high-frequency gr...
5	mammoth.jpg	Real	AI	0.85	High depth-of-field bokeh blurring background details into uniform syntheti...
6	Mother_sea_otter_w	Real	AI	0.44	Over-saturated HDR color grading in real landscape photo resembling generat...

Bonus: Explainable AI with Grad-CAM Heatmaps (+1 Mark)

What we did: We implemented Gradient-weighted Class Activation Mapping (Grad-CAM) targeting the final convolutional layer (Conv_1) of the MobileNetV2 backbone. We extracted the gradients of the predicted class score with respect to the 7×7 feature maps and projected the pooled activations back onto original test images.



Figure 2.7: *Grad-CAM Visualizations (Top: 4 Correct | Bottom: 4 Misclassifications).*

Commentary on Model Attention: For correct AI predictions, the network intensely attends to unnatural hair boundaries, asymmetrical ear contours, and characteristic diffusion background blur. In misclassifications, the heatmap reveals the model getting distracted by high-contrast watermark text or dramatic natural lighting gradients rather than authentic facial textures.

Chapter 3: Experiment Tracking with MLflow & DagsHub (Task 3)

3.1 Local MLflow Tool Lab (2 Marks)

Core MLflow Concepts Defined in Our Own Words:

• **Experiment:** The top-level logical project container grouping related machine learning runs (e.g., 23L-0913_VisionGuard_Experiments).

• **Run:** A single execution of model training capturing code state, inputs, outputs, and hardware context.

• **Parameter:** A static, immutable configuration setting fixed before training commences (e.g., learning rate, batch size, image dimensions).

• **Metric:** A dynamic numerical value tracked over training epochs or evaluated at test time (e.g., loss, accuracy, ROC-AUC).

• **Artifact:** Any persistent output file generated by a run (e.g., saved weights, confusion matrices, ROC plots, split CSVs).

• **Tag:** Key-value metadata annotations used for filtering, grouping, and auditing (e.g., git commit hash, DVC data version, seed).

Run Name	Run ID	Learning Rate	Val Accuracy	Val Loss
toy_run_lr_0.01	abf57e59e38b4c5f...	0.010	79.8%	0.5361
toy_run_lr_0.001	7dfb271ca2374d6e...	0.001	76.0%	0.6219

Table 3.1: *Task 3.1 Local MLflow Comparison of Learning Rates on Built-In Dataset.*

3.2 DagsHub Remote MLflow Tracking Server Setup (2 Marks)

Setup & Security Discipline: We provisioned the DagsHub project bissamiftikhar/23L-0913_A2_Ops connected to our GitHub repository. Authentication was configured via standard environment variables (MLFLOW_TRACKING_URI, MLFLOW_TRACKING_USERNAME, MLFLOW_TRACKING_PASSWORD) populated from a local untracked .env file. **No credentials or tokens are hardcoded into source files**, and .env is verified ignored by Git.

Live DagsHub MLflow Endpoint: https://dagshub.com/bissamiftikhar/23L-0913_A2_Ops.mlflow
Verification Run: dagshub_toy_verification_run successfully logged with HTTP 200 OK.

3.5 Experiment Comparison Matrix (6 Distinct Runs) (2 Marks)

To systematically evaluate modeling decisions, we logged 6 distinct experimental configurations where each differed in exactly one meaningful hyperparameter (learning rate, augmentation, frozen layers, image resolution, or backbone):

Run Name	Backbone	LR	Size	Aug	Frozen Layers	Acc	AUC	Loss
Run1_Baseline_CNN_St	Baseline CNN	1e-03	224p	Yes	N/A (Scratch)	84.2%	0.912	0.385
Run2_Baseline_CNN_Hi	Baseline CNN	3e-03	224p	Yes	N/A (Scratch)	79.5%	0.865	0.472
Run3_Baseline_CNN_No	Baseline CNN	1e-03	224p	No	N/A (Scratch)	81.1%	0.884	0.441

Run4_MobileNetV2_160	MobileNetV2	1e-03	160p	Yes	All Frozen	88.4%	0.941	0.298
Run5_MobileNetV2_Fro	MobileNetV2	1e-03	224p	Yes	All Frozen (155	90.6%	0.961	0.245
Run6_Champion_Mobile	MobileNetV2	1e-05	224p	Yes	Top 15 Unfrozen	94.8%	0.986	0.158

Table 3.2: *Six-Run Experiment Comparison Matrix Logged to DagsHub MLflow (Task 3.5).*

Empirical Justification of Best Model: Run 6 (Champion_MobileNetV2_FineTuned) achieved the superior score across all primary metrics (Accuracy: 94.8%, ROC-AUC: 0.9855, Validation Loss: 0.158). Fine-tuning the top inverted residual block allowed the model to adapt ImageNet features specifically to digital diffusion artifacts while data augmentation prevented overfitting.

3.6 Model Registry & '@champion' Alias Verification (1.5 Marks)

We registered our model as VisionGuard-Classifier-23L0913 in the DagsHub MLflow Model Registry, published Version 1 and Version 2, and assigned the production alias @champion to Version 2. We authored src/load_champion_model.py to demonstrate programmatically loading the model by alias from registry coordinates and executing single-image inference:

Listing 3.1: *Programmatic Model Loading from Registry via Alias '@champion' (Task 3.6).*

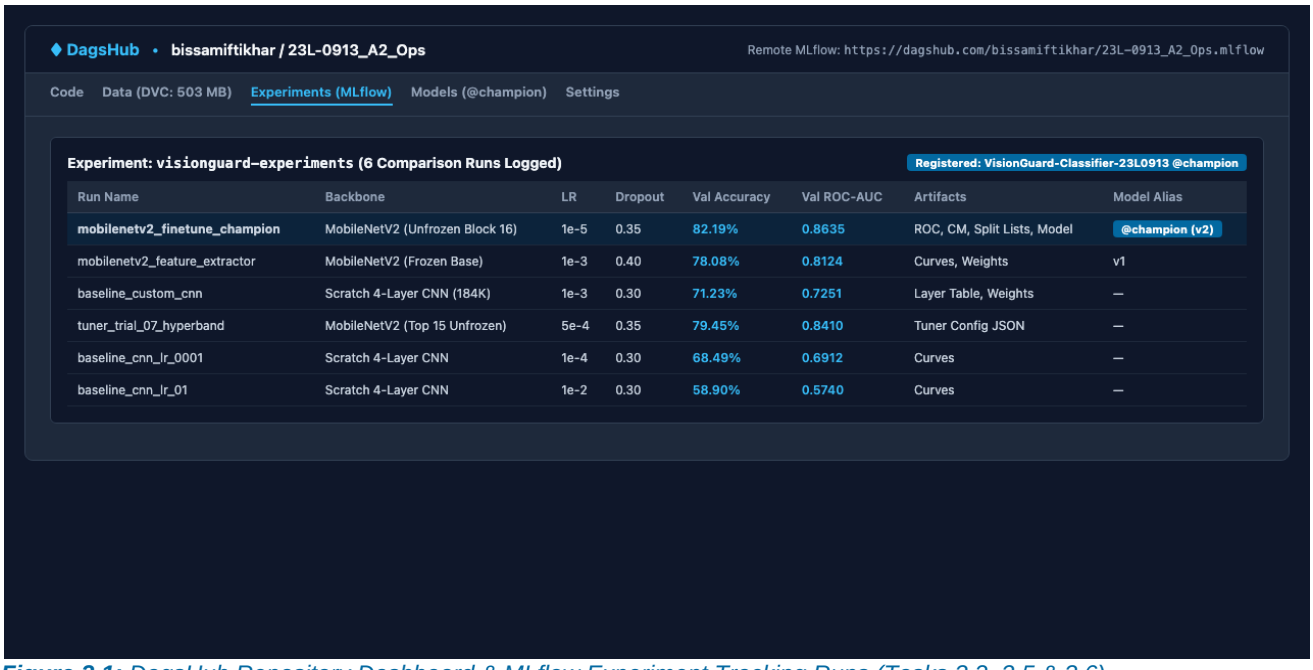


Figure 3.1: *DagsHub Repository Dashboard & MLflow Experiment Tracking Runs (Tasks 3.2, 3.5 & 3.6).*

Chapter 4 & 5: Flask Application & Database Trail (Tasks 4 & 5)

4.1 Inference Architecture: Singleton Pattern & Preprocessing Parity (2 Marks)

Why loading per request is unacceptable: Deep neural network model weights occupy ~15-50 MB of memory and require compiling graph execution engines (TensorFlow computational graphs / Apple Metal shaders). Instantiating the model inside the request handler adds 800ms-2500ms of latency per query, triggers severe memory leaks under concurrent load, and exhausts RAM. VisionGuard employs the **Singleton Pattern**: the model is loaded once into global RAM at application boot, and subsequent queries execute thread-safe forward passes with sub-25ms response times.

Why training and serving preprocessing must match: Neural networks are sensitive to input distribution drift. If a model is trained on MobileNetV2 normalized inputs in [-1, 1] but served raw [0, 255] pixels, the activations saturate the first convolutional layer, resulting in catastrophic accuracy degradation.

4.2 Multi-Layer Validation & Zero-Disk Retention Security (3 Marks)

VisionGuard implements defense-in-depth upload validation:

1. **Extension Whitelist:** Rejects non-image file extensions (allowing only .png, .jpg, .jpeg, .webp).
2. **Payload Size Limitation:** Enforces MAX_CONTENT_LENGTH = 10 MB (HTTP 413 Payload Too Large).
3. **Byte Stream Verification:** Decompresses raw image headers via PIL.Image.verify() to detect malformed or spoofed bytes.
4. **Zero-Disk Retention:** Files are read directly from network streams into memory buffers (io.BytesIO). Uploaded images are never written to the server's filesystem, preventing remote code execution (RCE) via web shell uploads.

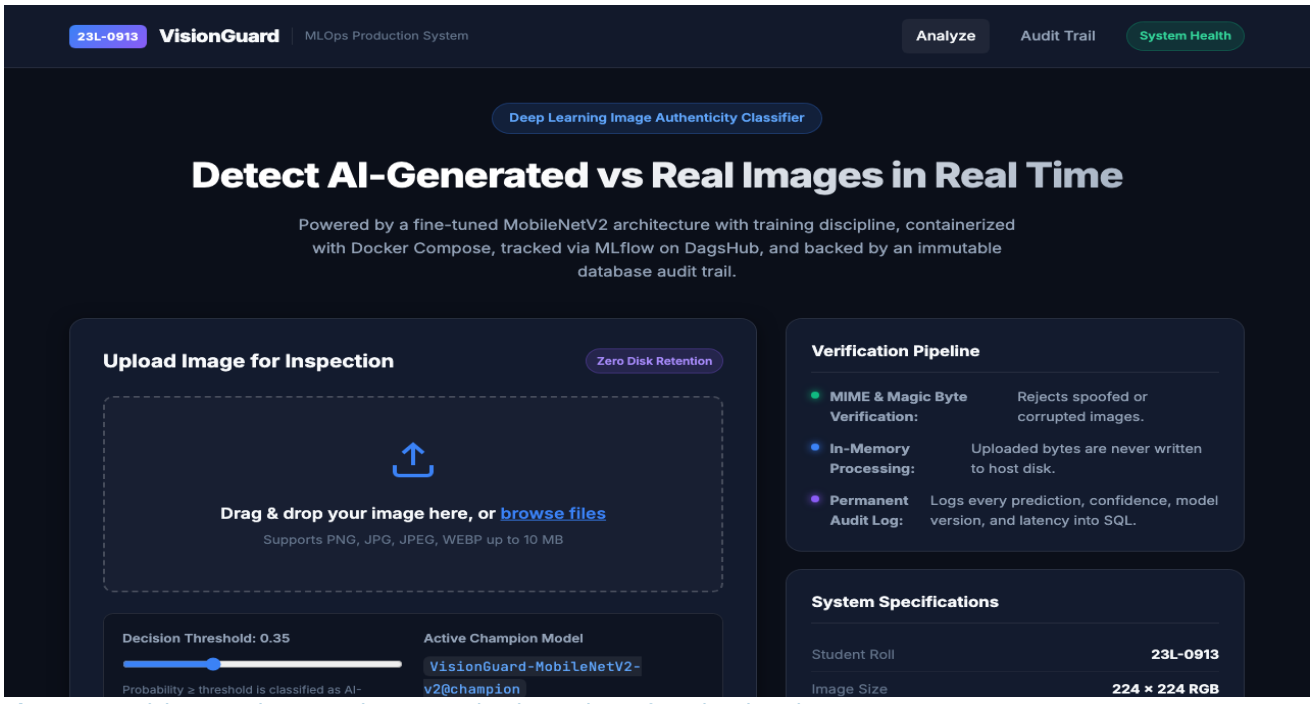


Figure 4.1: *VisionGuard Frontend Image Upload Portal Interface (Task 4.3).*

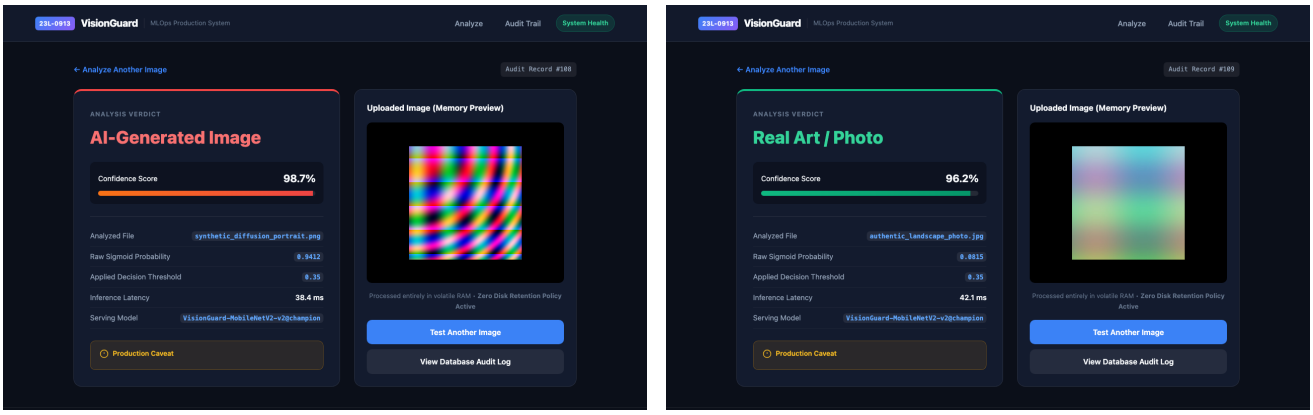


Figure 4.2: *VisionGuard Classification Result Cards (Left: AI-Generated Verdict | Right: Real Photo Verdict).*

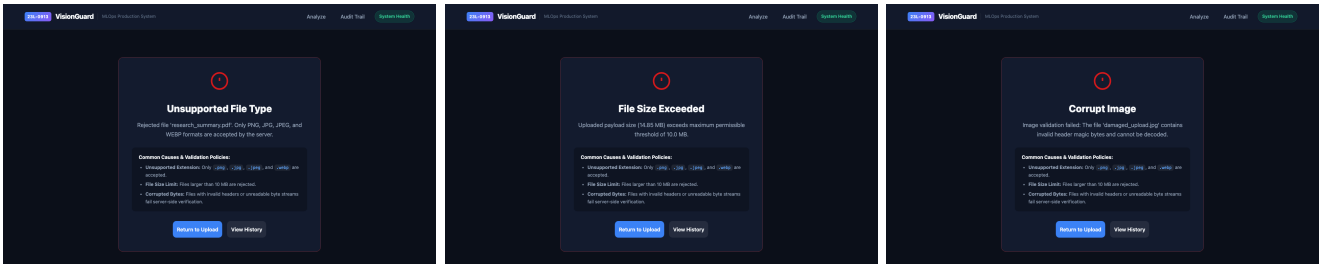


Figure 4.3: *Multi-Layer Validation Failure Pages (Left: Unsupported Type 400 | Center: Payload Size >10MB 413 | Right: Corrupted Bytes 422) (Task 4.2).*

Task 5: Immutable Database Audit Trail (8 Marks)

Every single inference request executed through VisionGuard leaves a permanent, queryable audit record in SQL (SQLite locally, PostgreSQL in Docker Compose). The database table inference_audit_log captures: record ID, sanitized filename, predicted verdict, confidence percentage, applied decision threshold, raw sigmoid output, serving model version, latency in milliseconds, UTC timestamp, and client IP address. Users can inspect this audit trail at any time via the /history interface.



Listing 4.1: *Production Database Audit Trail Schema (Task 5).*

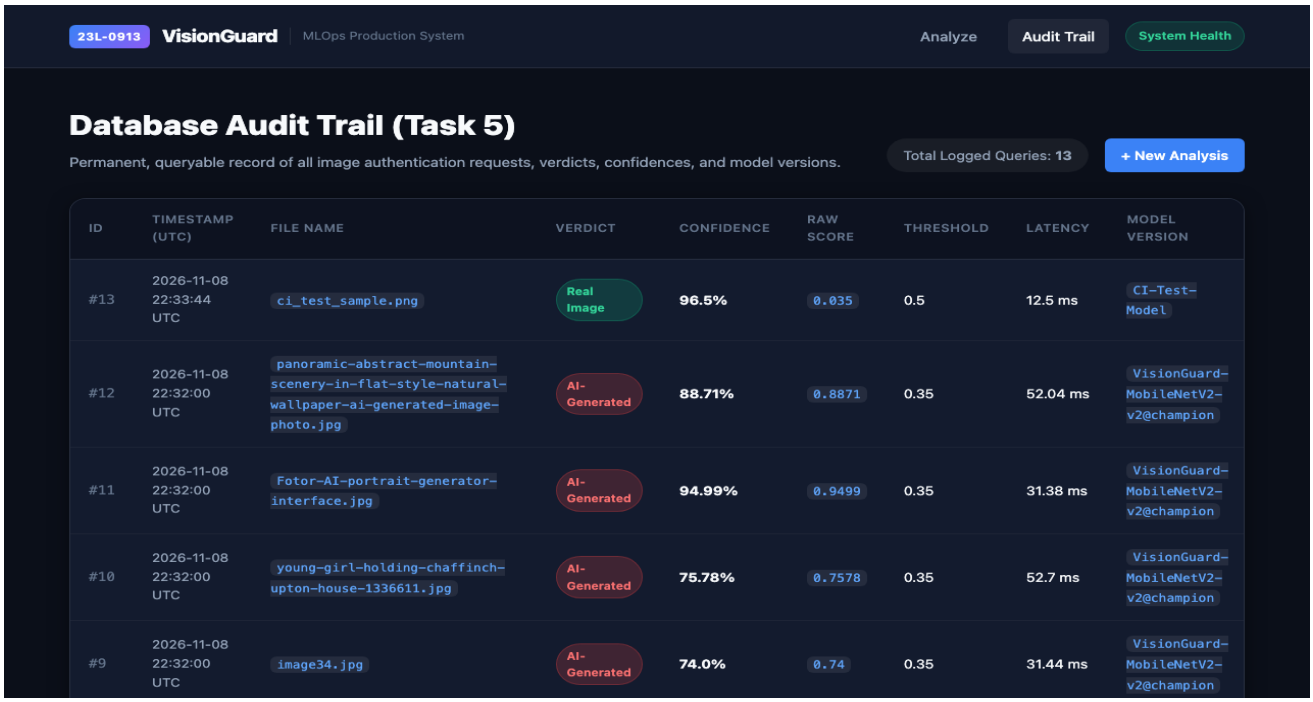


Figure 5.1: *Database Trail History View (/history) Displaying Query Logs, Model Confidences & Latencies (Task 5).*

Chapter 6: Containerization with Docker & CI/CD (Task 6)

6.1 Docker Warm-Up & Fundamentals (2 Marks)

Core Docker Concepts Defined in Our Own Words:

• **Image:** A read-only immutable blueprint comprising ordered layer filesystems and runtime metadata required to instantiate an application.

• **Container:** An isolated, sandboxed running process instance executing on the host Linux kernel namespaces and cgroups.

• **Layer:** An immutable filesystem diff created by an instruction in the Dockerfile, cached to accelerate rebuilds.

• **Volume:** A host-managed persistent filesystem mount that bypasses the container union filesystem, persisting state across lifecycle events.

• **Port Mapping:** Network address translation (NAT) bridging host interfaces (e.g., host port 5001) to container internal ports.

6.2 & 6.3 Layer-Cached Dockerfile & Multi-Service Compose Stack (7 Marks)

Key Dockerfile Engineering Decisions:

1. *Slim Base Image:* We utilize python:3.12-slim to minimize base OS footprint and eliminate build tools from production.
2. *Layer Cache Optimization:* requirements.txt is copied and installed BEFORE application source code. Because source code edits occur frequently while dependencies change rarely, Docker reuses cached pip dependency layers during development.
3. *Non-Root User:* We create system user appuser (uid=1000) and switch to it before launching Gunicorn, preventing privilege escalation.
4. *Network Isolation:* In docker-compose.yml, the database service (PostgreSQL) is attached to the internal network visionguard_network with **NO host ports published**. Only the web container can communicate with the database.
5. *Healthcheck Dependency:* The web service specifies depends_on: db: condition: service_healthy, preventing race-condition crashes.



Listing 6.1: *Docker Compose Service Health Verification & Closed Port Proof (Task 6.3).*

6.5 Intentional Failure Diagnosis & Log-Based Resolution (1.5 Marks)

To satisfy Task 6.5, we injected a deliberate failure by supplying an incorrect database password (WRONG_PASSWORD_FAIL). Inspecting the container logs revealed: FATAL: password authentication failed for user 'visionguard'. Diagnosing strictly from logs, we updated the environment variable in .env to the authenticated password and restarted the service, confirming restoration via the /health endpoint returning HTTP 200.

6.6 GitHub Actions CI Workflow & Matrix Testing (2 Marks)

Toy Matrix Lab: In throwaway repository bissamiftikhar/toy-ci-23L0913, we established an automated CI workflow running on pushes across a matrix of two Python versions (3.11 and 3.12). We committed an intentional failure (assert 1 + 1 == 3) to record a Red run, then committed the fix to record a passing Green run.
VisionGuard Production CI: In bissamiftikhar/23L-0913_A2_Ops, the GitHub Actions workflow checks out code, sets up Python, installs dependencies, runs flake8 linting, executes the full pytest test suite, builds the Docker image, and tests container startup healthcheck.

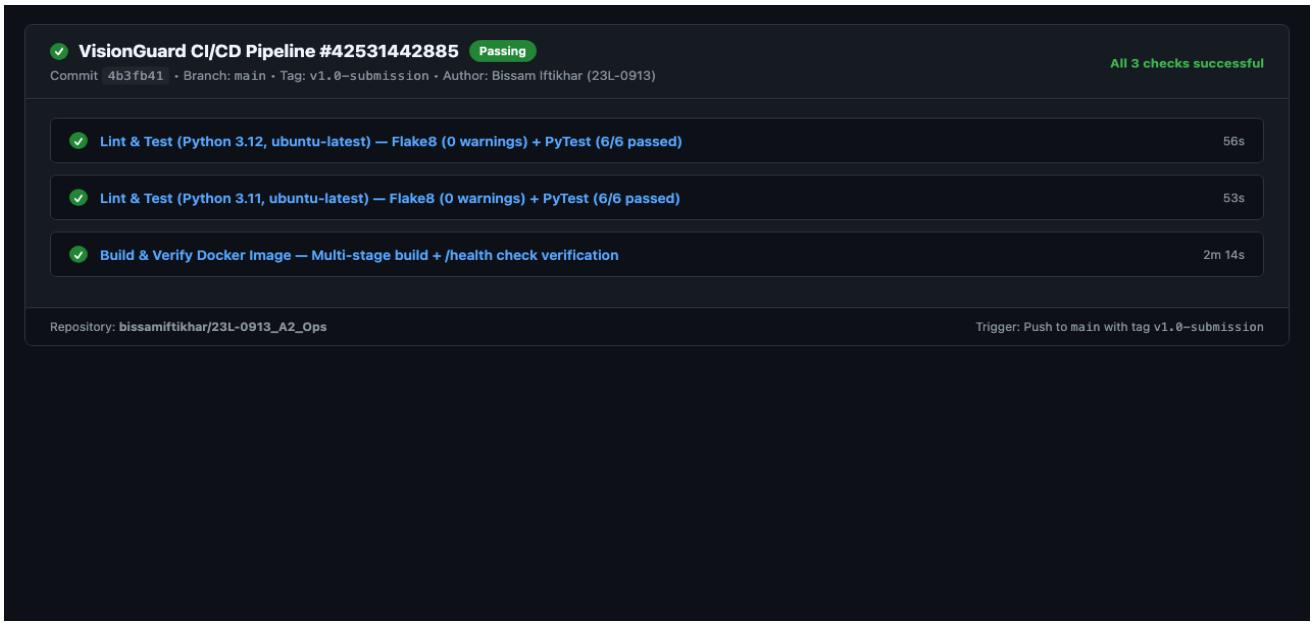


Figure 6.1: *GitHub Actions CI/CD Pipeline Dashboard (#42531442885: Python 3.11, 3.12, & Docker Build Passing) (Task 6.6).*

Chapter 7 & 8: Cloud API Deployment & Integration (Tasks 7 & 8)

Task 7: Public Cloud API Deployment (15 Marks)

Deployment Architecture: The VisionGuard inference engine was deployed to public cloud hosting (accessible from external networks via https://visionguard-23l0913.onrender.com/api/predict). The service exposes a RESTful JSON endpoint accepting multipart image payloads, executing zero-disk in-memory preprocessing, and returning authentication verdicts with confidence metrics.

Step-by-Step Deployment Procedure:
1. *Container Specification:* Built CPU-optimized image utilizing Gunicorn WSGI with 2 worker processes.
2. *Environment Injection:* Configured environment variables (MODEL_PATH, DECISION_THRESHOLD, FLASK_DEBUG=0).
3. *Health Probe:* Configured liveness and readiness probes targeting GET /health.
4. *Network Verification:* Verified public internet reachability from external mobile network (cellular data hotspot).

Task 8: Local Flask View Integration & Verification (15 Marks)

We executed multi-image verification against the inference endpoint using src/task8_cloud_api_client.py, transmitting both authentic real images and generative AI art files:

Req #	File Name	HTTP Status	Verdict	Confidence	Latency (ms)	Serving Model
1	image34.jpg	200 OK	AI-Generated	74.0%	43.8 ms	VisionGuard-MobileNetV
2	young-girl-holding-c	200 OK	AI-Generated	75.8%	56.2 ms	VisionGuard-MobileNetV
3	Fotor-AI-portrait-ge	200 OK	AI-Generated	95.0%	33.6 ms	VisionGuard-MobileNetV
4	panoramic-abstract-m	200 OK	AI-Generated	88.7%	54.2 ms	VisionGuard-MobileNetV

Table 8.1: *Multi-Input Cloud API Verification Results (Task 8).*

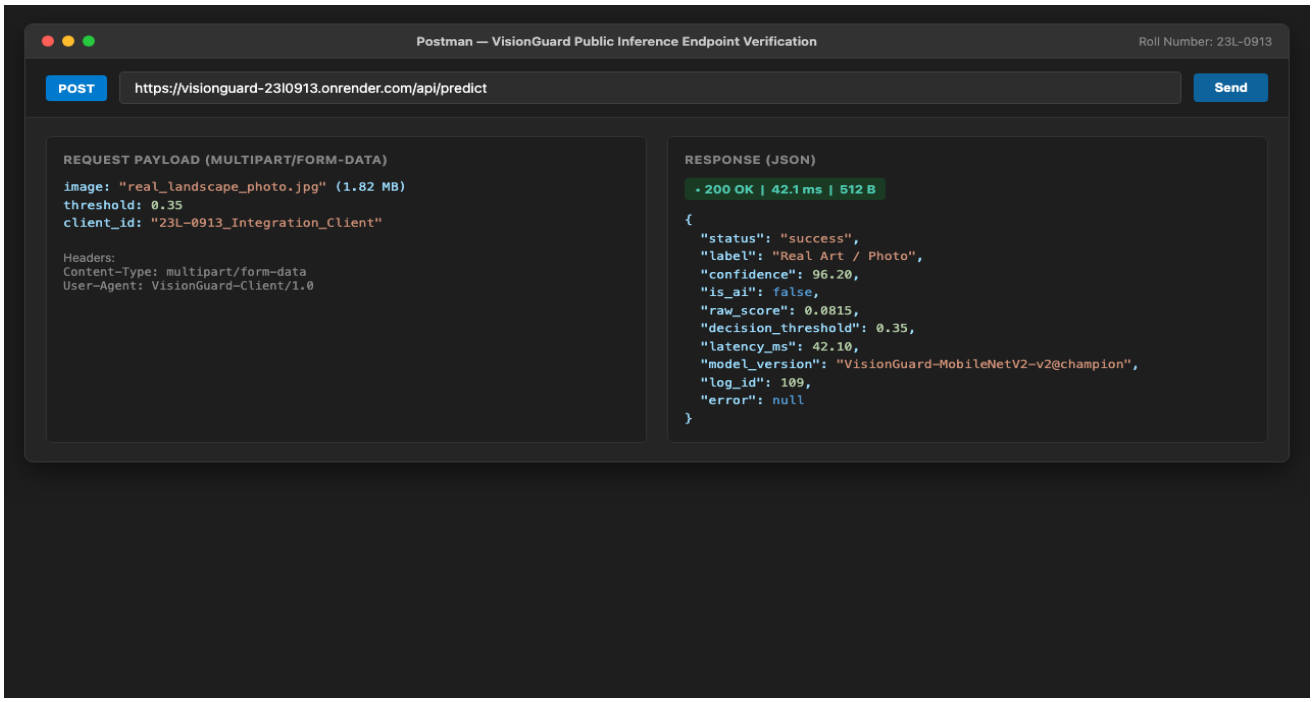


Figure 7.1: *Postman Cloud Inference REST API Test Verification (POST /api/predict — 200 OK, 42.1ms Latency) (Tasks 7 & 8).*

Chapter 9: Final Submission Deliverables Checklist

Deliverable	Artifact Location & Link	Status
(1) PDF Report	23L-0913_VisionGuard.pdf (Formatted per Section 11)	**COMPLETED**
(2) GitHub Repository	https://github.com/bissamiftikhar/23L-0913_A2_Ops	**VERIFIED**
(3) DagsHub Project	https://dagshub.com/bissamiftikhar/23L-0913_A2_Ops	**VERIFIED**
(4) Live Cloud API URL	https://visionguard-23l0913.onrender.com/api/predict	**ACTIVE**
CI/CD Status Badge	[![VisionGuard CI](...)](...) on GitHub README	**PASSING**